

## **UNIT 2**

### **Software Requirement Specification**

#### **What is Requirement:-**

Requirements are descriptions of the services that a software system must provide and the constraints under which it must operate.

Requirements can range from high-level abstract statements of services or system constraints to detailed mathematical functional specifications

Requirements Engineering is the process of establishing the services that the customer requires from the system and the constraints under which it is to be developed and operated

Requirements may serve a dual function:

As the basis of a bid for a contract

As the basis for the contract itself

#### **Types of Requirement:-**

##### **User requirements**

Statements in natural language plus diagrams of the services that the systems provides and its operational constraints.

Written for customers

##### **System requirements**

A structured document setting out detailed descriptions of the system services.

Written as a contract between client and contractor

##### **Software specification**

A detailed software description which can serve as a basis for a design or implementation.

Written for developers

#### **SRS (Software Requirement Specification):-**

- 1.The user should be provided with facilities to define the type of external files
2. Each external file type may have an associated tool which may be applied to the file
3. Each external file type may be represented as a specific icon on the user's

display

4. Facilities should be provided for the icon representing an external file to be defined by the user

5. When a user selects an icon representing an external file, the effect of that selection is to apply the tool associated with the type of external file to the file represented by the selected icon

### **Functional, non-functional and domain requirements**

#### **Functional requirements**

Statements of services that the system should provide, how the system should react to particular inputs and how the system should behave in particular situations

#### **Non-functional requirements**

Constraints on the services or functions offered by the system such as timing constraints, constraints on the development process, standards, etc.

#### **Domain requirements**

Requirements that come from the application domain of the system that reflect the characteristics of that domain.

### **Software Engineering Benefits:-**

1. **Improved Quality:** By following established software engineering principles and techniques, the software can be developed with fewer bugs and higher reliability.
2. **Increased Productivity:** Using modern tools and methodologies can streamline the development process, allowing developers to be more productive and complete projects faster.
3. **Better Maintainability:** Software that is designed and developed using sound software engineering practices is easier to maintain and update over time.
4. **Reduced Costs:** By identifying and addressing potential problems early in the development process, software engineering can help to reduce the cost of fixing bugs and adding new features later on.
5. **Increased Customer Satisfaction:** By involving customers in the development process and developing software that meets their needs, software engineering can help to increase customer satisfaction.

6. **Better Team Collaboration:** By using Agile methodologies and continuous integration, software engineering allows for better collaboration among development teams.
7. **Better Scalability:** By designing software with scalability in mind, software engineering can help to ensure that software can handle an increasing number of users and transactions.
8. **Better Security:** By following the [Software Development Life Cycle \(SDLC\)](#) and performing security testing, software engineering can help to prevent security breaches and protect sensitive data.

### **Role of Management in Software Development: -**

**1)People** – When we refer to people in software development, we actually mean hierarchy of roles in software development such as managers, team leaders, senior developers etc.

They are ones who form development team and work in development of software. The order of priority of people among other dependency factors is **first** (highest).

Software development is people-centric activity. Software development requires good managers, people who understand psychology of people and provide good leadership. A good manager does not ensure success of project, but can increase probability of success.

**2)Product** – A product is solution of problem that is handed to user who is struggling with problem. The priority of product among other dependency factors is **second** that is after **people**.

The product or solution to problem is constructed based on requirements, which are objectives and scope work of problem. This helps managers to select best approach within constraints imposed by delivery deadlines, budgetary restrictions, personnel availability, technical interfaces etc.

**3)Process** – The process is way in which we produce software. It provides framework from which comprehensive plan for software development can be established. If process is weak, end product will undoubtedly suffer.

The priority of process among other dependency factors is **third**, after **people** and **product**, however, it plays very critical role for success of project. There are many life cycle models and process improvements models.

**4)Project** – A proper planning is required to monitor status of development and to control complexity. For project to be successful, it should be within budget, follow policies and requirements defined, and completed within timeline stated.

Most of projects overrun cost by 100% of budget and are not delivered on time. The priority of project among other dependency factors is **last**(lowest).

### **Role of Metrics and Measurement:-**

**Software Measurement:** A measurement is a manifestation of the size, quantity, amount, or dimension of a particular attribute of a product or process. Software measurement is a titrate impute of a characteristic of a software product or the software process.

### **Software Measurement Principles**

1. **Formulation:** The derivation of software measures and metrics appropriate for the representation of the software that is being considered.
2. **Collection:** The mechanism used to accumulate data required to derive the formulated metrics.
3. **Analysis:** The computation of metrics and the application of mathematical tools.
4. **Interpretation:** The evaluation of metrics results in insight into the quality of the representation.
5. **Feedback:** Recommendation derived from the interpretation of product metrics transmitted to the software team.

### **Need for Software Measurement**

Software is measured to:

- Create the quality of the current product or process.
- Anticipate future qualities of the product or process.
- Enhance the quality of a product or process.
- Regulate the state of the project concerning budget and schedule.

- Enable data-driven decision-making in project planning and control.
- Identify bottlenecks and areas for improvement to drive process improvement activities.
- Ensure that industry standards and regulations are followed.

### **Classification of Software Measurement**

There are 2 types of software measurement:

1. **Direct Measurement:** In direct measurement, the product, process, or thing is measured directly using a standard scale.
2. **Indirect Measurement:** In indirect measurement, the quantity or quality to be measured is measured using related parameters i.e. by use of reference.

### **Software Metrics**

A metric is a measurement of the level at which any impute belongs to a system product or process.

Software metrics are a quantifiable or countable assessment of the attributes of a software product. There are 4 functions related to software metrics:

1. **Planning**
2. **Organizing**
3. **Controlling**
4. **Improving**

### **Characteristics of software Metrics**

1. **Quantitative:** Metrics must possess a quantitative nature. It means metrics can be expressed in numerical values.
2. **Understandable:** Metric computation should be easily understood, and the method of computing metrics should be clearly defined.

3. **Applicability:** Metrics should be applicable in the initial phases of the development of the software.
  4. **Repeatable:** When measured repeatedly, the metric values should be the same and consistent.
  5. **Economical:** The computation of metrics should be economical.
  6. **Language Independent:** Metrics should not depend on any programming language.
1. **Product Metrics:** Product metrics are used to evaluate the state of the product, tracing risks and undercover prospective problem areas. The ability of the team to control quality is evaluated. Examples include lines of code, cyclomatic complexity, code coverage, defect density, and code maintainability index.
  2. **Process Metrics:** Process metrics pay particular attention to enhancing the long-term process of the team or organization. These metrics are used to optimize the development process and maintenance activities of software. Examples include effort variance, schedule variance, defect injection rate, and lead time.
  3. **Project Metrics:** The project metrics describes the characteristic and execution of a project. Examples include effort estimation accuracy, schedule deviation, cost variance, and productivity. Usually measures-
    - Number of software developer
    - Staffing patterns over the life cycle of software
    - Cost and schedule
    - Productivity

### **Advantages of Software Metrics**

1. Reduction in cost or budget.
2. It helps to identify the particular area for improvising.
3. It helps to increase the product quality.
4. Managing the workloads and teams.

5. Reduction in overall time to produce the product,.
6. It helps to determine the complexity of the code and to test the code with resources.
7. It helps in providing effective planning, controlling and managing of the entire product.

### **Disadvantages of Software Metrics**

1. It is expensive and difficult to implement the metrics in some cases.
2. Performance of the entire team or an individual from the team can't be determined. Only the performance of the product is determined.
3. Sometimes the quality of the product is not met with the expectation.
4. It leads to measure the unwanted data which is wastage of time.
5. Measuring the incorrect data leads to make wrong decision making.

### **SYSTEM DESIGN**

- The design activity begins when requirement document for the software to be developed is available .
  - Designing is the first step in moving from the problem domain towards the solution domain .
  - Design is essential bridge between requirement specification and the final solution for satisfying the requirement.
  - Designing is used in two ways
  - 1)verb 2) noun
  - Verb-it represents the process of design
  - Noun-it represents the result of the design process which is the design for the system.
  - Design of the system is basically a blue print or plan for solution for system
  - The goal of the design process is to produce a model or representation of a system which can be used later to build the system the produced model is called design of the system.

- System design models may be divided into two classes.

### 1)static models

These models represent aspect of software that changed during execution

### 2)dynamic models

- These models represent aspect software changed during execution

## **DESIGN METHODOLOGY**

- A design methodology is a systematic approach to create a design by applying of a set of techniques and guidelines, most design methodology focuds on the system design.
- A design can be object-oriented or fuctional-oriented.
- In object oriented design ,the model is the design represents data abstraction .
- In fuctional oriented design, the design consist of the module definition ,with each module supporting a fuctional abstraction.
- The design process for system has two levels .

### **1)top level design or system**

If focus on the designing which modules are needed for the system and specification of this module,how the module should be inter concepted call the top level designing or system designing.

### **2)detail design or logical design**

- The internal design for the module or how the specification of the modules can be satisfied is decided call detail design or logical design .
- Detail design is the extension of the system design .

## **PRINCIPLE OF THE DESIGN**

- 01)design of the system is correct .
- 02)desigb should clearly verifiable complete& traceable.
- 03)simplicity means creating simple design
- 04)design should uni forunity and integration .
- 05)design should be structure to accommodate change.

## **SOFTWARE DESIGN STRATEGY**



- Understand the problem
- Generate possible solution to the problem.
- Evaluate the possible solution to pick the best.
- Unsure that the solution is sufficient ,and that the work is complete and well documented.

## DESIGN PATTERN

### WHAT IS DESIGN PATTERN?

- Pattern is a name of inside which conversion the sense of prove solution to receiving problem within contain contact.
- It desirable design structure that solve the particular design problem within specified context if forces that may have import of the manner in which the pattern is apply or use.
- It provide the description that enable to designer that.
- 01)whether the pattern is applicable or not for the current work .
- 02)whether the pattern can be reuse or not.
- 03)pattern can be serving a guides for developing .

### Characterized an effective design pattern in the following.

- **It solve a problem** pattern capture solution .not just abstract principles or strategies.
- **It is a proven concept** pattern capture solution with a track record , not theories or assumption
- **It describes a relationship** pattern don't just describe models .but describe deeper system structure and mechanisms.
- **The patterns has significant human component**

### TYPE OF DESIGN PATTERN

- **1)Architectural pattern** describe broad based design problem that are solve using a structure approach
- **2)data patterns** describe returning data oriented problem and the data modeling solution that can be used to solve them .
- **3)component patterns** components pattern also referred to as design pattern address problem associated with the development of sub system and of

components ,the manner in which they communicate with one another ,and their placement within a large architecture.

- **4)interface design patterns** describe common users interface problem and their solution with a system of forces that includes the specific characteristics of and users
- **5)web application pattern** web application pattern address a problem set that is encountered when building web application and often incorporates many of the other pattern categories just mentioned
- **6)creational pattern** focus on the creation composition and representation of object
- **7)behavioral patterns** behavioral pattern address problem associated with the assignment of responsibility between object and the manner in which communicate is effected between object.

## **BECOMING MASTER DESIGNER**

### **HOW BECOMING THE MASTER DESIGNER**

- One must learn the rules
- --algorithem,datastructure and language.
- One must learn the princip
- --structure programming moduler programming,object oriented

### **One must study the designs of other master.**

- --design pattern must be understood memonzed an applied
- There is thousands of exisping design pattern
- Most good designer have found solution to a variety of problem
- From trial and error.they now what a solution does and does not work
- They can related new problems to problems they have handled previously
- The problem has to find a way to formalize and generalize this now ledge so that it can be reused as fast as possible.

## **IMPLEMENTING A DESLGN**

- ->after the design phase is,next step is the implementation of the solution.

- ->for large and/or complex system involving server design teams,the design must be integrated and verified.
- ->object oriented design, obviously is best implemented using an object oriented programming language.
- ->the design must be modified considering the particular of the implementation language.
- ->design goes from abstract concepts to low level considerations.
- ->Lower level considerations can have an impact on the higher design levels.
- ->when the design is implemented some parts of the design have to be adapted to the particulars of the implementation language.
- ->try to implement a bad design often leads to project failure.
- ->designers and implementation must work hand in hand for smooth transition between design and implementation.

### **EVALUATING A DESIGN**

- An evaluation of software design is the continuous process for evaluating the design,number of common characteristics are.
- The design must be complete.
- The design must be simple.
- System for the conversion of the requirement model into design representation.
- Document for representing functional component and interface.

### **PROBLEM PARTITIONING**

- The complexity of large problem and the limitations of human mind do not allow large problem to be care for a huge monoliths.
- For solving the large problem, the basic principle is the time tested the “divide and conquer.
- It means the problem can be divided in two small pieces and each piece can be word separately.
- The main goal of the partition is the divided problem in manageable small piece and each piece can be small separately.

- The partition is also use for simplicity and understandability of design.
- The maintenance is minimize if each peace part in the system can easily related to the application and each peace can be modify separately.
- Problem partition is also aims for design verification.
- It is essential for solving the complex problem, it ideas to hierarchy design.
- It can be represented as hierarchy of component.

### **ABSTRACTION**

- When you consider a modular solution to any problem them many level of abstraction can be pass.
- An abstraction of a component describe the external behavior of a that component without bothering with the internal detail .
- An abstraction definition of component is simpler when the component it self .
- Abstraction is indispensable of the design process ;it is essentially for problem partition
- The components are not isolate from each other ,they interact with each rather & designers how to specify how the components are interact with each other
- It is also use for existing component place the important role in maintain on phase because to modify the system the first step understand “when the system does and how”
- There are two common methods for abstraction for software system
- Function abstraction
- Data abstraction
  - 1)**function abstraction**
  - The modules specify by the the function or it specify
  - **Example:-**the module to compute the log of value.it ca be abstractly by function log
  - To short input array can be represented by the function shorting function
  - 2)**data abstraction**
  - It is any entity in the real world
  - It provides some services to the environment to which it belong
  - The entities provide some fix predefine service

- STRATEGY OF DESIGN
- The strategy of design is
- **01)bottom up design**
- **02)top-down design**
- **03)hybrid design**
- **01)bottom up design**
- A bottom up design approach start with designing the most basic or primitive components and proceeds to higher-level components that use these lower level component
- Bottom up methods work with layers of abstraction
- Starting from the very bottom.operational that provide a layer of abstraction are implemented.
- **02)top-down design**
- The top-down design approach starts from the higher level component of the hierarchy and proceeds through to lower levels .
- While a bottom up approach starts with the lower level component of the hierarchy and proceeds through progressively higher level to the top level component.
- A top down design approach starts by identifying the major component of the system ,decomposing them into their lower level component and iterating unit the desired level of detail is achieved .
- Top down design methods often result in some form of stepwise refinement
- **03)hybrid design**
- Hybrid design is a combination of top down and bottom up design

### **FUNCTION ORIENTED VERSES OBJECT ORIENTED**

- **1)function oriented**
- The function oriented design ,the design consist of modules definition .
- Each modules support functional abstraction.
- The system is a view as transformation functional means transforming the input in to the need output .
- It is follow the

- 01)the definition of all the major data structure in the system.
- 02)all the major modules of the system .
- 03)how the modules interact with each other
  
- **2)OBJECT ORIENTED**
- In the object oriented ,the modules in the design represented the data abstraction
- The basic system component is a modules that support the data abstraction the main goal of design ,is to identity the object that the system contains and the relationship and interaction between object
- The design is to partition the system in to know of object whether then partition is to know of function.
- The finale system is represented in the terms of object and classes .which define relationship between object classes .